

# **Taller Fundamentos de Algoritmia, C# y Preparación de Entorno .NET**

**Karol Diley Galvin Gonzalez**

**Ficha:  
2922211**

**C# Y .NET**

**SENA CENTRO DE SERVICIOS FINANCIEROS**

## A. Variables en algoritmos

### A1. 5 preguntas (concepto)

1. ¿Qué diferencia hay entre variable y constante en un algoritmo? Da un ejemplo de cada una.

Una variable puede cambiar su valor durante la ejecución del programa.

Una constante mantiene siempre el mismo valor, no se puede modificar después de haber sido definida. Pero ambas se usan para almacenar datos, pero sirven para propósitos diferentes. Usamos variables cuando necesitamos cambiar el valor (por ejemplo, una nota, un contador, etc.), y constantes cuando queremos proteger un valor que no debe cambiar (por ejemplo, el valor de PI, o un porcentaje fijo de IVA).

2. Explica el concepto de tipo de dato y por qué importa al declarar variables.

Un tipo de dato es una forma de clasificar la información que una variable puede almacenar y es importante porque permite al programa saber cuánto espacio necesita en memoria, evita errores, ya que no puedes hacer operaciones incorrectas (como sumar texto con números). y facilita el control del comportamiento de las variables (por ejemplo, redondeos en números decimales o mayúsculas en texto).

3. ¿Qué problema puede causar convertir un string a int sin validar? Da un caso.

Si intentas convertir un texto (string) a un número entero (int) sin validar antes que ese texto realmente sea un número, el programa puede fallar y lanzar un error en tiempo de ejecución. Específicamente, puede generar una excepción como `FormatException` en C#.

4. ¿Cuál es la diferencia entre asignación e igualdad? Ejemplifícalo.

La asignación se usa para darle un valor a una variable por ejemplo; `int edad = 25;` y la igualdad se usa para comparar dos valores y saber si son iguales. Devuelve un valor booleano (`true` o `false`), por ejemplo; `int edad = 25;`  
`bool esIgual = (edad == 30);` .

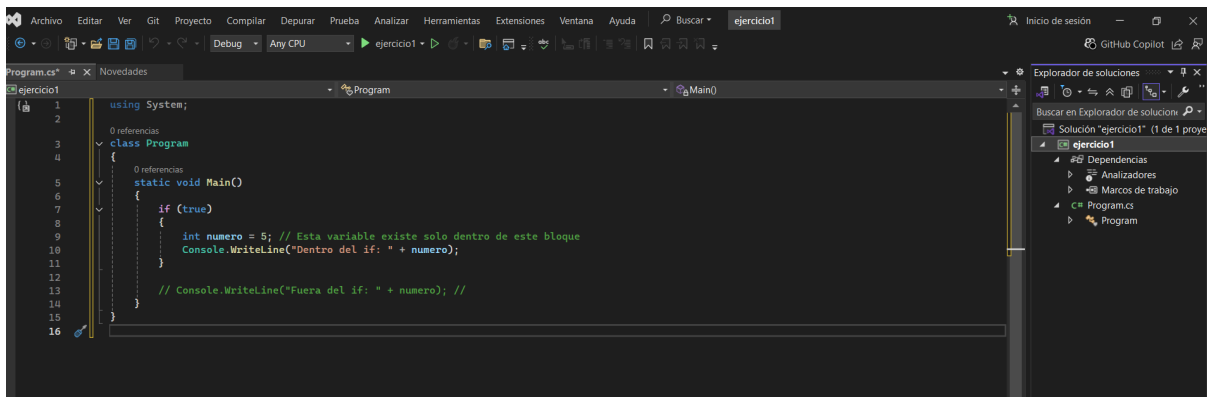
5. ¿Qué es el alcance (scope) de una variable? Ejemplo donde una variable no es visible.

Es el lugar del programa donde esa variable existe y se puede usar. Fuera de ese espacio, la variable no es visible ni accesible.

En C#, el alcance de una variable depende del bloque de código donde se declara, como por ejemplo dentro de un método, un `if`, un `for`, etc.

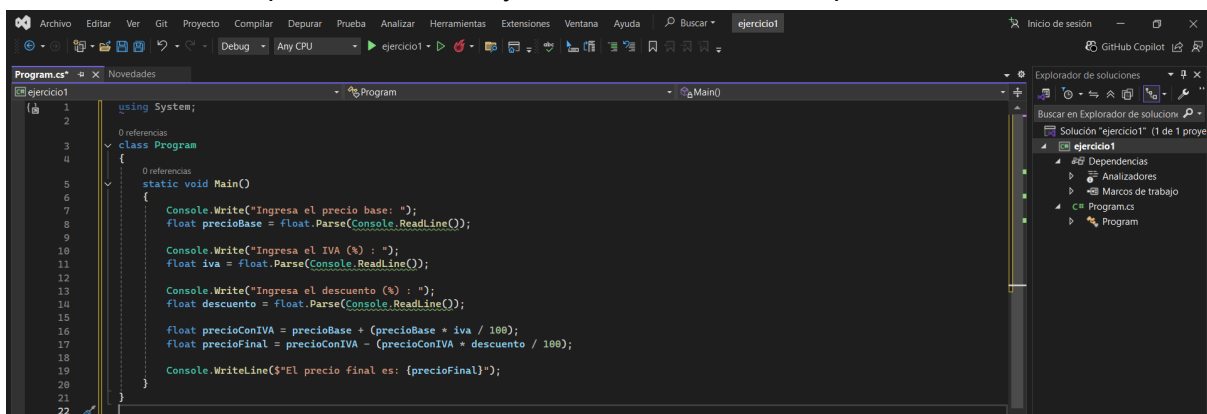
## A2. 5 ejercicios (práctica)

6. Conversión: Pide (o define) una temperatura en °C y calcula °F. Fórmula:  $F = C * 9/5 + 32$ .



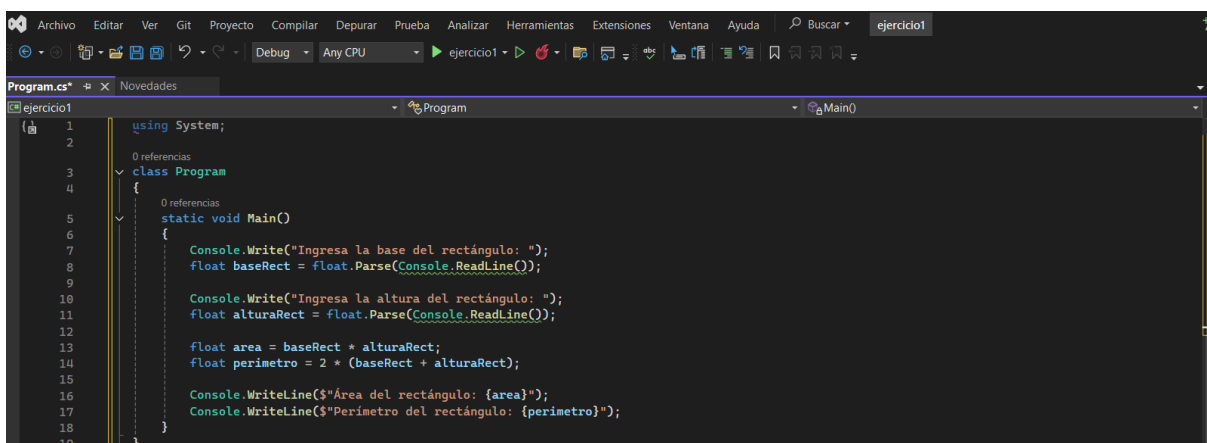
```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         if (true)
8         {
9             int numero = 5; // Esta variable existe solo dentro de este bloque
10             Console.WriteLine("Dentro del if: " + numero);
11
12             // Console.WriteLine("Fuera del if: " + numero); //
13         }
14     }
15 }
16
```

7. Costo final: Dado precioBase, iva% y descuento%, calcula el precio final.



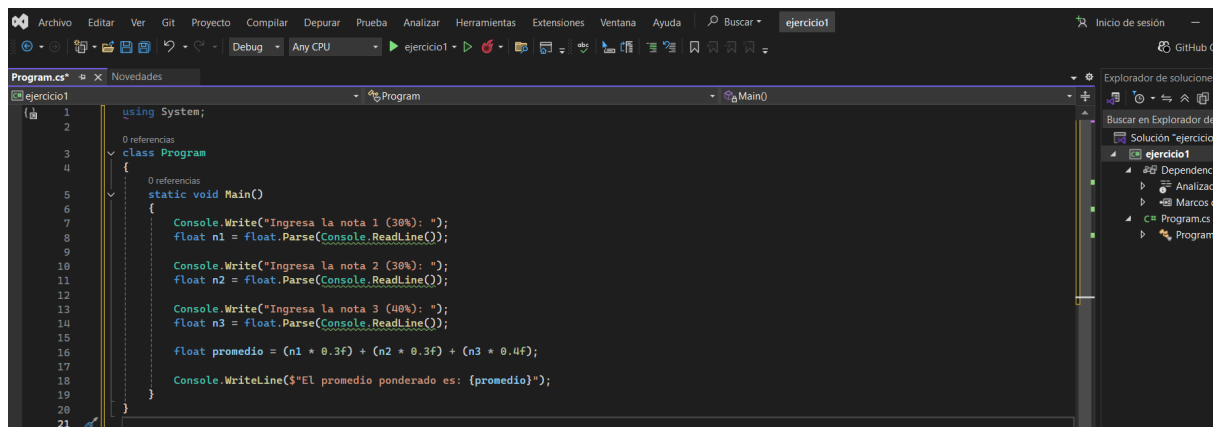
```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         Console.WriteLine("Ingresa el precio base: ");
8         float precioBase = float.Parse(Console.ReadLine());
9
10        Console.WriteLine("Ingresa el IVA (%): ");
11        float iva = float.Parse(Console.ReadLine());
12
13        Console.WriteLine("Ingresa el descuento (%): ");
14        float descuento = float.Parse(Console.ReadLine());
15
16        float precioConIVA = precioBase + (precioBase * iva / 100);
17        float precioFinal = precioConIVA - (precioConIVA * descuento / 100);
18
19        Console.WriteLine($"El precio final es: {precioFinal}");
20    }
21 }
22
```

8. Área y perímetro: Con base y altura (float), calcula área y perímetro de un rectángulo. using System;



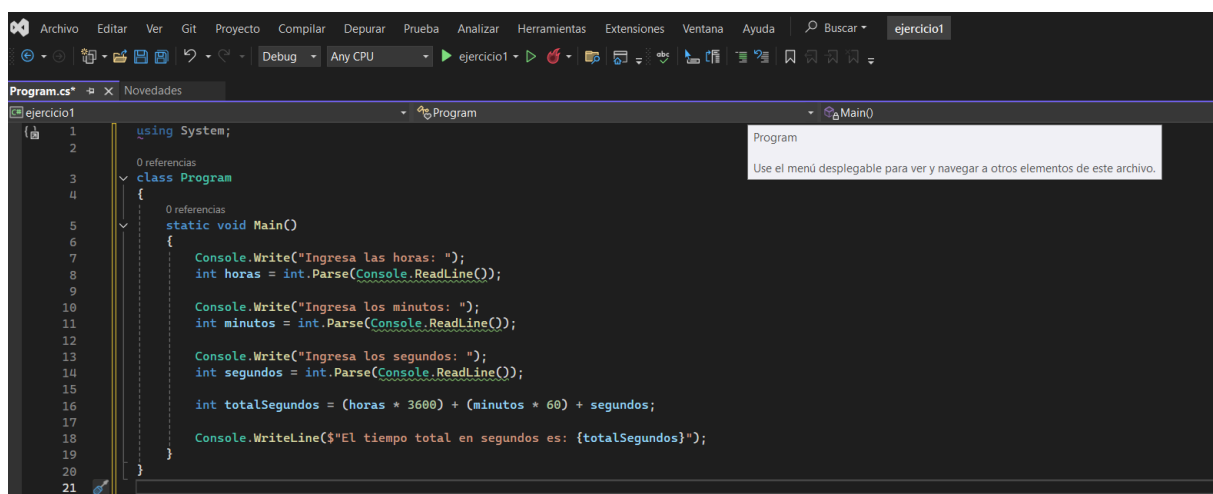
```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         Console.WriteLine("Ingresa la base del rectángulo: ");
8         float baseRect = float.Parse(Console.ReadLine());
9
10        Console.WriteLine("Ingresa la altura del rectángulo: ");
11        float alturaRect = float.Parse(Console.ReadLine());
12
13        float area = baseRect * alturaRect;
14        float perimetro = 2 * (baseRect + alturaRect);
15
16        Console.WriteLine($"Área del rectángulo: {area}");
17        Console.WriteLine($"Perímetro del rectángulo: {perimetro}");
18    }
19 }
```

9. Promedio ponderado: Con notas n1 (30%), n2 (30%), n3 (40%), calcula el promedio.



```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         Console.WriteLine("Ingresa la nota 1 (30%): ");
8         float n1 = float.Parse(Console.ReadLine());
9
10        Console.WriteLine("Ingresa la nota 2 (30%): ");
11        float n2 = float.Parse(Console.ReadLine());
12
13        Console.WriteLine("Ingresa la nota 3 (40%): ");
14        float n3 = float.Parse(Console.ReadLine());
15
16        float promedio = (n1 * 0.3f) + (n2 * 0.3f) + (n3 * 0.4f);
17
18        Console.WriteLine($"El promedio ponderado es: {promedio}");
19    }
20 }
21
```

10. Tiempo total: Dadas horas, minutos y segundos (enteros), conviértelos a segundos totales.



```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         Console.WriteLine("Ingresa las horas: ");
8         int horas = int.Parse(Console.ReadLine());
9
10        Console.WriteLine("Ingresa los minutos: ");
11        int minutos = int.Parse(Console.ReadLine());
12
13        Console.WriteLine("Ingresa los segundos: ");
14        int segundos = int.Parse(Console.ReadLine());
15
16        int totalSegundos = (horas * 3600) + (minutos * 60) + segundos;
17
18        Console.WriteLine($"El tiempo total en segundos es: {totalSegundos}");
19    }
20 }
21
```

## B. Estructuras de control (condicionales)

### B1. 5 preguntas (concepto)

11. ¿Cuándo usar if/else y cuándo switch? Da un ejemplo de cada uno.

**IF AND ELSE:** Se usa cuando se tiene condiciones complejas o comparaciones que involucran operadores lógicos (AND, OR), o cuando la condición a evaluar no se puede simplificar a una única variable o valor.

**if/else:** Cuando tienes condiciones más complejas, con comparaciones y operadores lógicos (AND, OR) o diferentes tipos de expresiones.

```
int edad = 20;
if (edad >= 18 && edad < 65)
    Console.WriteLine("Adulto");
else
    Console.WriteLine("No adulto");
```

**switch:** Cuando tienes una variable que quieres comparar con varios valores concretos, es más limpio y fácil de leer.

```

int opcion = 2;
switch (opcion)
{
    case 1:
        Console.WriteLine("Opción 1");
        break;
    case 2:
        Console.WriteLine("Opción 2");
        break;
    default:
        Console.WriteLine("Opción no válida");
        break;
}

```

12. Explica la diferencia entre > y >= con un caso donde importa.

> significa "mayor que", excluye el valor igual.

>= significa "mayor o igual que", incluye el valor igual.

```

int nota = 70;
if (nota > 70)
    Console.WriteLine("Aprobó con nota mayor a 70");
else
    Console.WriteLine("No aprobó o sacó exactamente 70");

if (nota >= 70)
    Console.WriteLine("Aprobó con nota 70 o más");

```

13. ¿Por qué es mala idea anidar demasiados if? Propón una alternativa.

Porque el código se vuelve difícil de leer, entender y mantener.

Alternativa: usar **switch**, **guard clauses** o separar en funciones más pequeñas.

14. ¿Qué es un guard clause y para qué sirve?

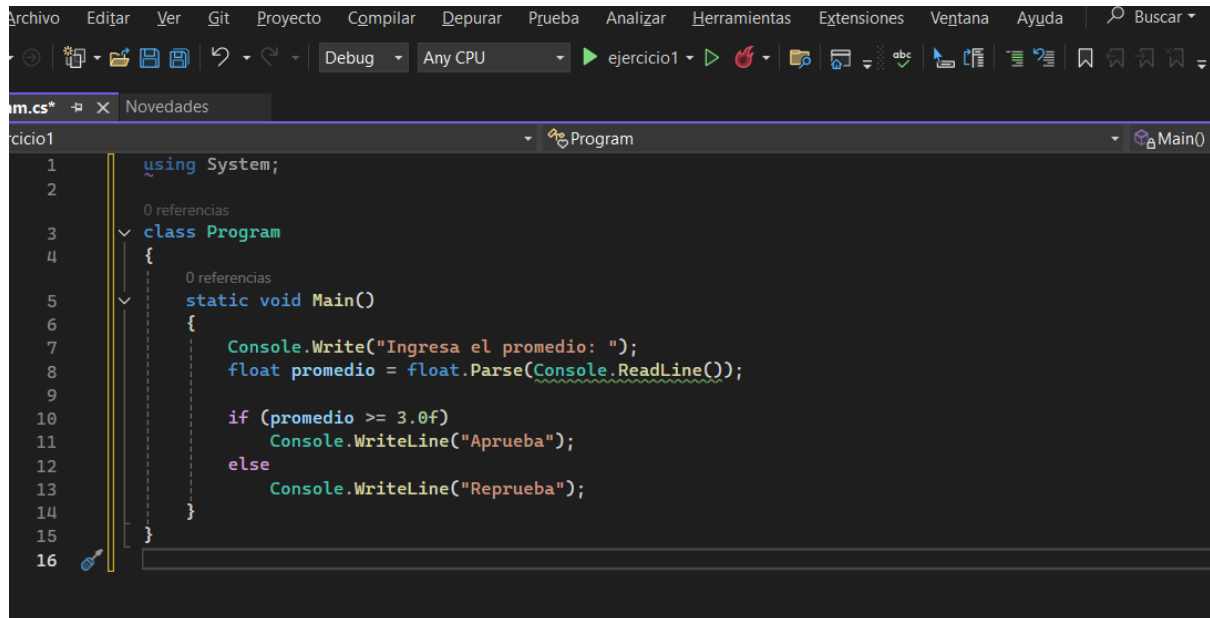
Es una condición al inicio de una función o bloque que **detiene la ejecución si no se cumple algo**, para evitar anidar código y mejorar la legibilidad.

15. ¿Qué retorna una expresión condicional (operador ternario) y cuándo conviene usarla? todo en .net por favor

Retorna un valor dependiendo de una condición, todo en una línea, y conviene usarla para asignaciones rápidas o cuando la condición es simple.

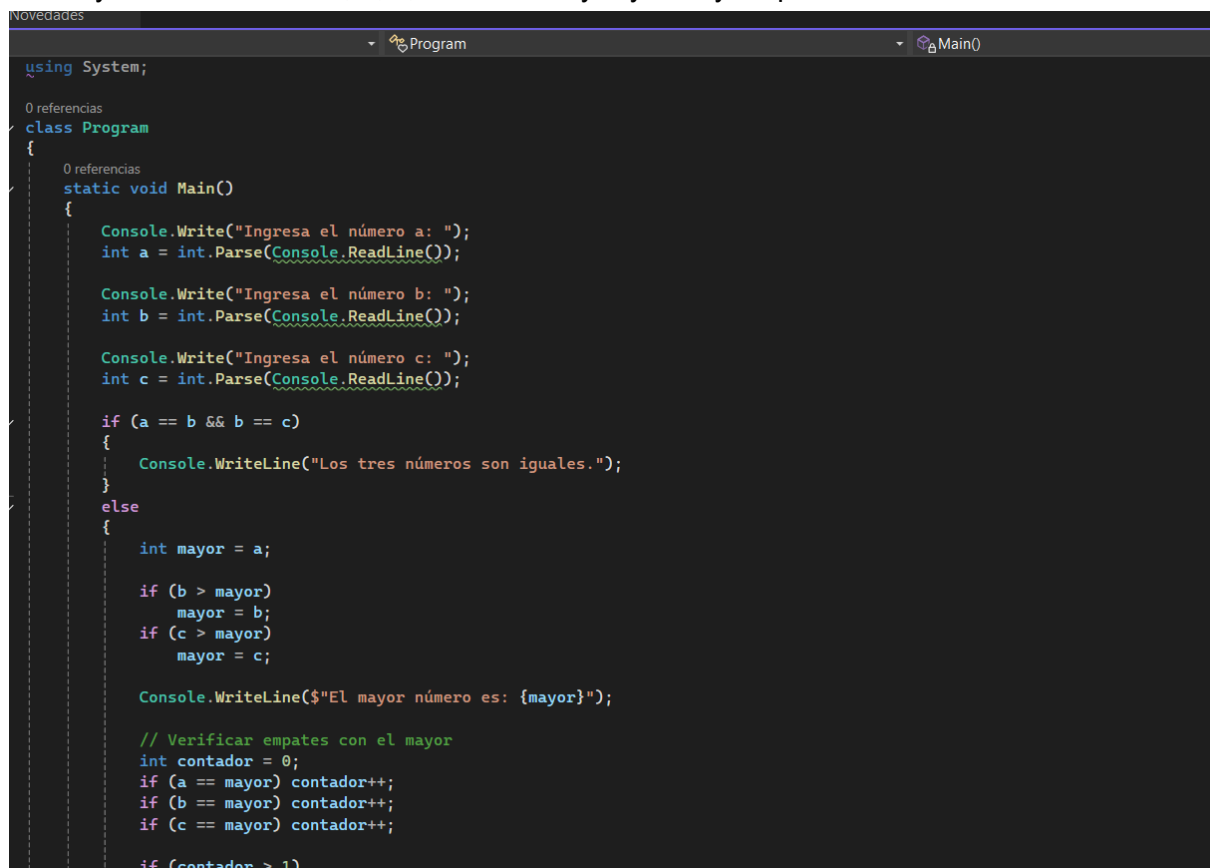
## B2. 5 ejercicios (práctica)

16. Aprobado/Reprobado: Si promedio  $\geq 3.0$  "Aprueba", si no "Reprueba". → →



```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         Console.Write("Ingresa el promedio: ");
8         float promedio = float.Parse(Console.ReadLine());
9
10        if (promedio >= 3.0f)
11            Console.WriteLine("Aprueba");
12        else
13            Console.WriteLine("Reprueba");
14        }
15    }
16 }
```

17. Mayor de tres: Dado a, b, c, indica el mayor y si hay empates.



```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Ingresa el número a: ");
        int a = int.Parse(Console.ReadLine());

        Console.Write("Ingresa el número b: ");
        int b = int.Parse(Console.ReadLine());

        Console.Write("Ingresa el número c: ");
        int c = int.Parse(Console.ReadLine());

        if (a == b && b == c)
        {
            Console.WriteLine("Los tres números son iguales.");
        }
        else
        {
            int mayor = a;

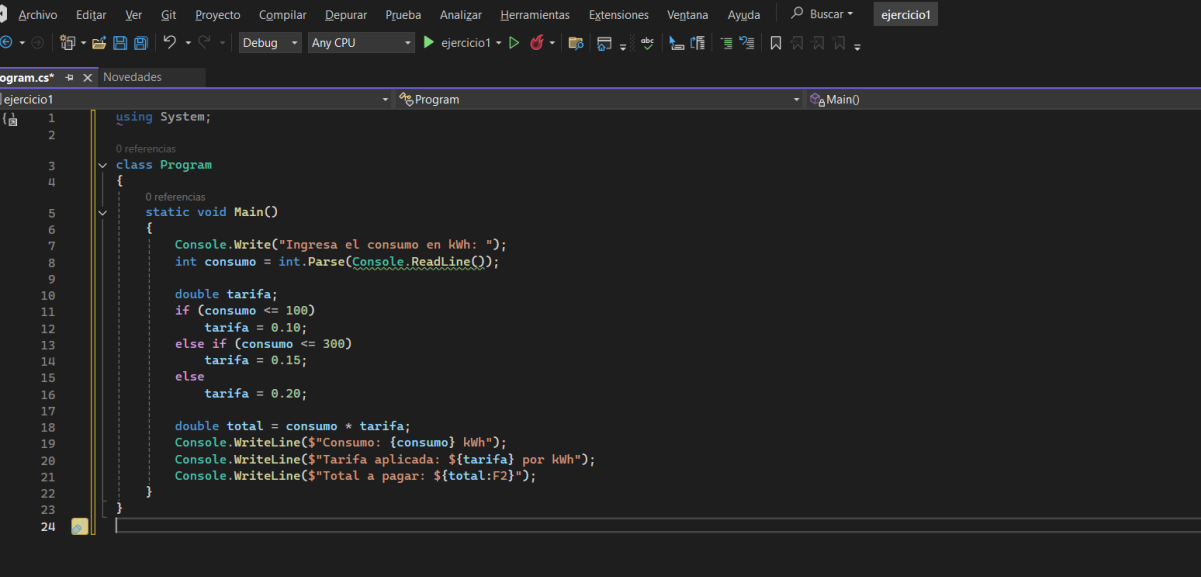
            if (b > mayor)
                mayor = b;
            if (c > mayor)
                mayor = c;

            Console.WriteLine($"El mayor número es: {mayor}");

            // Verificar empates con el mayor
            int contador = 0;
            if (a == mayor) contador++;
            if (b == mayor) contador++;
            if (c == mayor) contador++;

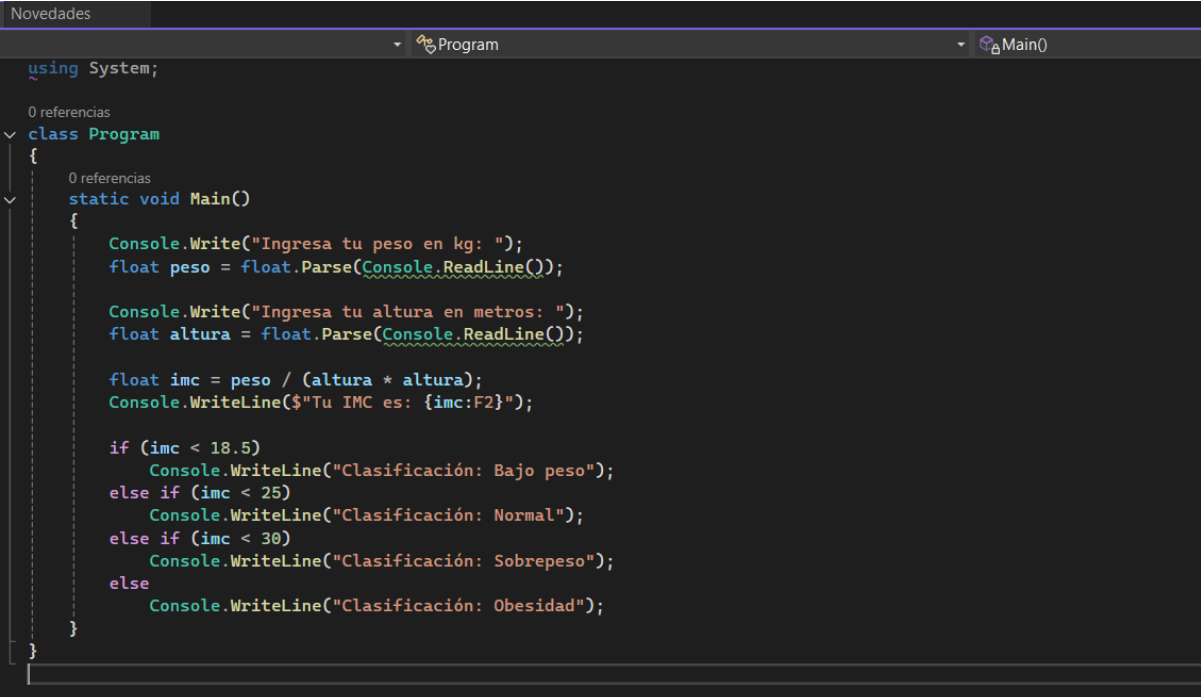
            if (contador > 1)
```

18. Tarifa eléctrica: Si consumo kWh  $\leq$  100 tarifa A;  $\leq$  300 tarifa B;  $>$  300 tarifa C. Muestra valor a pagar.



```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         Console.Write("Ingresa el consumo en kWh: ");
8         int consumo = int.Parse(Console.ReadLine());
9
10        double tarifa;
11        if (consumo <= 100)
12            tarifa = 0.10;
13        else if (consumo <= 300)
14            tarifa = 0.15;
15        else
16            tarifa = 0.20;
17
18        double total = consumo * tarifa;
19        Console.WriteLine($"Consumo: {consumo} kWh");
20        Console.WriteLine($"Tarifa aplicada: ${tarifa} por kWh");
21        Console.WriteLine($"Total a pagar: ${total:F2}");
22    }
23 }
24
```

19. Clasificación IMC: Calcula IMC y clasifica (bajo peso, normal, sobrepeso, obesidad) según rangos OMS.



```
using System;

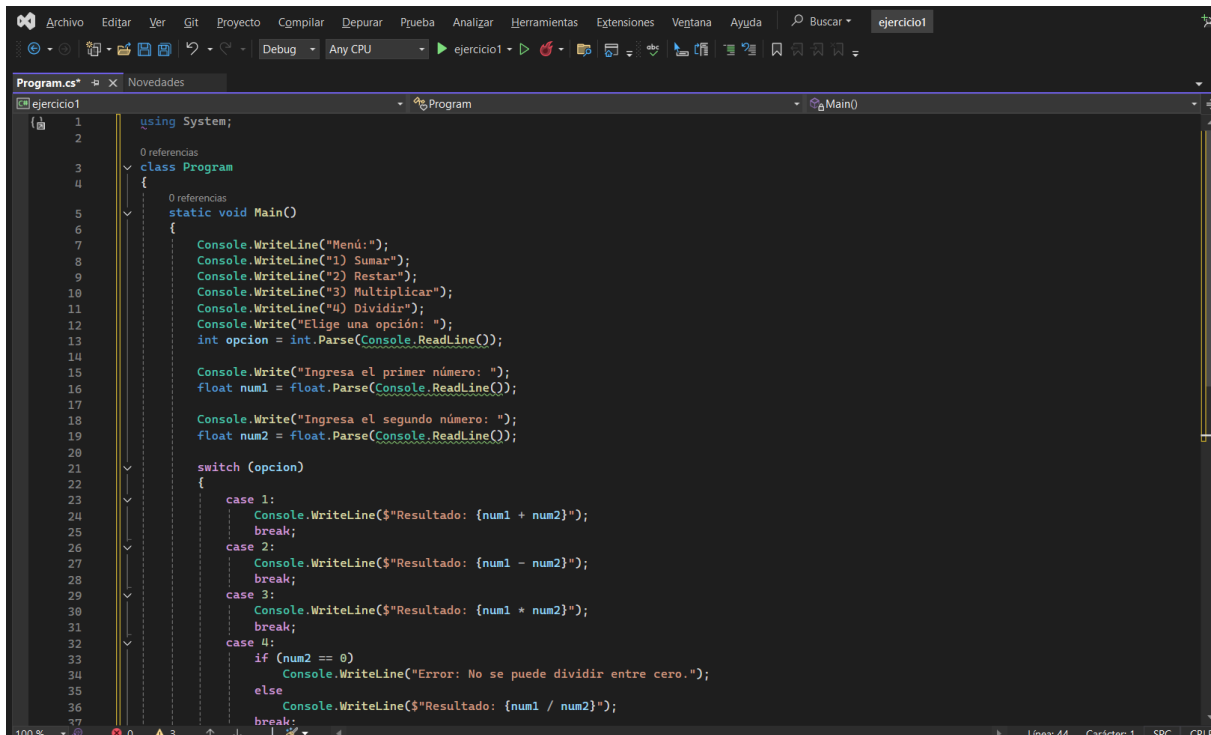
class Program
{
    static void Main()
    {
        Console.Write("Ingresa tu peso en kg: ");
        float peso = float.Parse(Console.ReadLine());

        Console.Write("Ingresa tu altura en metros: ");
        float altura = float.Parse(Console.ReadLine());

        float imc = peso / (altura * altura);
        Console.WriteLine($"Tu IMC es: {imc:F2}");

        if (imc < 18.5)
            Console.WriteLine("Clasificación: Bajo peso");
        else if (imc < 25)
            Console.WriteLine("Clasificación: Normal");
        else if (imc < 30)
            Console.WriteLine("Clasificación: Sobrepeso");
        else
            Console.WriteLine("Clasificación: Obesidad");
    }
}
```

20. Menú con switch: 1) Sumar, 2) Restar, 3) Multiplicar, 4) Dividir. Lee opción y opera dos números. Valida división por cero.



```
1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         Console.WriteLine("Bienvenido");
8         Console.WriteLine("1) Sumar");
9         Console.WriteLine("2) Restar");
10        Console.WriteLine("3) Multiplicar");
11        Console.WriteLine("4) Dividir");
12        Console.WriteLine("Elige una opción: ");
13        int opcion = int.Parse(Console.ReadLine());
14
15        Console.WriteLine("Ingresa el primer número: ");
16        float num1 = float.Parse(Console.ReadLine());
17
18        Console.WriteLine("Ingresa el segundo número: ");
19        float num2 = float.Parse(Console.ReadLine());
20
21        switch (opcion)
22        {
23            case 1:
24                Console.WriteLine($"Resultado: {num1 + num2}");
25                break;
26            case 2:
27                Console.WriteLine($"Resultado: {num1 - num2}");
28                break;
29            case 3:
30                Console.WriteLine($"Resultado: {num1 * num2}");
31                break;
32            case 4:
33                if (num2 == 0)
34                    Console.WriteLine("Error: No se puede dividir entre cero.");
35                else
36                    Console.WriteLine($"Resultado: {num1 / num2}");
37                break;
38        }
39    }
40 }
```

C1. 5 preguntas (concepto)

21. Diferencia entre for, while y do-while (caso de uso de cada uno).

- *for*: Usas cuando sabes cuántas veces repetir.

```
for (int i = 0; i < 10; i++)
```

```
{
```

```
    Console.WriteLine(i);
```

```
}
```

- *while*: Repite mientras la condición sea verdadera.

```
int i = 0;
```

```
while (i < 10)
```

```
{
```

```
    Console.WriteLine(i);
```

```
    i++;
```

```
}
```



- *do-while*: Igual que while, pero se ejecuta al menos una vez.

```
int i = 0;
do
{
    Console.WriteLine(i);
    i++;
} while (i < 10);
```

22. ¿Qué es una condición de salida y qué pasa si no existe?

Una **condición de salida** es la regla o situación que hace que un bucle termine.

Si no existe o no funciona bien, el bucle **se ejecuta para siempre**, lo que puede hacer que el programa se quede "pegado" o que se bloquee.

23. ¿Cuándo usar un contador y cuándo un acumulador? Ejemplo.

**Contador**: Cuenta cuántas veces ocurre algo (número de veces).

**Acumulador**: Suma valores o junta cosas (total o suma).

```
int contador = 0;
int acumulador = 0;

for (int i = 1; i <= 5; i++)
{
    if (i % 2 == 0)
        contador++;    // Cuenta cuántos números pares hay
    acumulador += i;    // Suma todos los números del 1 al 5
}

Console.WriteLine($"Pares: {contador}");    // Salida: 2
Console.WriteLine($"Suma total: {acumulador}"); // Salida: 15
```

24. ¿Qué hacen break y continue? Ejemplo breve de cada uno.

**break:** Sale inmediatamente del bucle, sin seguir más vueltas.

**continue:** Salta la vuelta actual y pasa a la siguiente iteración.

**Ejemplo:**

```
for (int i = 1; i <= 5; i++)  
{  
    if (i == 3)  
        break; // Sale del bucle cuando i es 3  
  
    Console.WriteLine(i);  
}  
  
// Salida: 1 2
```

```
for (int i = 1; i <= 5; i++)  
{  
    if (i == 3)  
        continue; // Salta imprimir el 3, sigue con el 4 y 5  
  
    Console.WriteLine(i);  
}  
  
// Salida: 1 2 4 5
```

25. ¿Por qué evitar bucles con complejidad  $O(n^2)$  si no es necesario?.

Porque un bucle con  **$O(n^2)$**  hace muchas más operaciones a medida que crece la entrada (por ejemplo, 10 elementos = 100 operaciones, 1000 elementos = 1,000,000 operaciones).

Esto **hace que el programa sea lento** y consuma más recursos.

Siempre es mejor buscar soluciones con menos pasos (como  $O(n)$ ) para que el programa sea más rápido y eficiente.

